



DATABASE DESIGN, MANAGEMENT, AND IMPLEMENTATION

Project



TUTOR: DR. A.J. FOFANAH

STUDENT ID: 28328: 19742

MSC. Computer Science

TABLE OF CONTENTS

DECLARATION	2
1.0 ABSTRACT.....	3
2.0 INTRODUCTION AND SCHEMA DESIGN RATIONALE.....	4
3.0 NORMALISATION PROOFS (QUESTION 1.1).....	5
3.1 First Normal Form (1NF).....	5
3.2 Second Normal Form (2NF)	5
3.3 Third Normal Form (3NF)	5
3.4 BCNF Discussion.....	6
3.5 Why Normalisation Matters Here	6
4.0 ER MODEL DIAGRAM	7
5.0 QUERY DESIGN DECISIONS AND RESULTS	8
5.1 Aggregation and Join Logic.....	8
5.2 Window Functions	8
5.3 Recursive and Set-Based Queries	8
5.4 Results Summary.....	9
6.0 INDEX ANALYSIS: BEFORE/AFTER STATISTICS	10
6.1 Indexes Added	10
6.2 Rationale	10
6.3 Before/After Discussion.....	10
7.0 TRANSACTION AND CONCURRENCY DISCUSSION (QUESTION 5.2).....	13
7.1 Atomic Transfer Logic.....	13
7.2 Deadlock Demonstration	13
7.3 Resolution Strategy	13
8.0 ADMINISTRATION AND SECURITY APPROACH.....	14
8.1 Backup and Recovery	14
8.2 Role-Based Security.....	14
8.3 Row-Level Security	14
8.4 Monitoring and Maintenance	14
9. CONCLUSION AND REFLECTIONS.....	16
10. REFERENCES	17

DECLARATION

By submitting this assignment, I certify that this work is solely our own. All SQL scripts are original or properly attributed. I have not shared my solution with any other student. I understand the consequences of academic dishonesty as outlined in the University's student conduct policy.

Student ID:

Date: 12 June, 2026.

1.0 ABSTRACT

This report presents the design and implementation of the UniHospital database project built in SQL Server. The system models core hospital operations, including departments, doctors, wards, patients, appointments, admissions, prescriptions, bills, and staff. The schema was designed to balance clarity, normalization, and querying. The project includes schema creation, seed data, analytical SQL queries, programmability objects, indexing strategies, transaction handling, and administration features such as backup, roles, and row-level security. Normalization was applied to reduce redundancy and improve update consistency, while foreign key constraints preserved referential integrity across clinical and financial workflows. Query design emphasized readable joins, grouped reporting, window functions, common table expressions, and set-based logic. Index choices were aligned to the most frequent filter and ordering patterns in the workload. Concurrency scripts illustrated deadlock risk and transaction ordering, while the administration section demonstrated recovery planning and access control. Overall, the project shows a realistic and extensible hospital database implementation.

2.0 INTRODUCTION AND SCHEMA DESIGN RATIONALE

The project implements a medium-sized hospital information system called 'UniHospital'. Its purpose is to support operational data management and analytical reporting in a single relational design. The schema starts from stable business entities and then connects them through carefully constrained transactional tables.

The main design rationale was:

- ❖ Keep operational entities separate: “Department”, “Doctor”, “Ward”, “Patient”, “Medication”, and “Staff”.
- ❖ Model clinical events explicitly: “Appointment”, “Admission”, and “Prescription”.
- ❖ Separate financial tracking through “Bill”.
- ❖ Preserve referential integrity with foreign keys instead of application-only assumptions.
- ❖ Use surrogate primary keys for consistent joins and simpler SQL.

The schema also reflects realistic workflow direction:

- ❖ A doctor belongs to one department.
- ❖ A ward belongs to one department.
- ❖ A patient can have many appointments and admissions.
- ❖ Prescriptions depend on both the admission context and the prescribing doctor.
- ❖ Bills may be linked directly to a patient and optionally to a specific admission.

This design supports transactional correctness first, then reporting. That is why the model favors narrow tables with clear responsibilities instead of denormalized wide records.

3.0 NORMALISATION PROOFS (QUESTION 1.1)

The schema is normalized primarily to Third Normal Form, with several tables also satisfying BCNF under the project assumptions.

3.1 First Normal Form (1NF)

All relations satisfy 1NF because:

Every table has a primary key.

- All attributes are atomic.

- There are no repeating groups or array-like columns for departments, medications, or appointments.

Examples:

“Patient(PatientID, FirstName, LastName, DOB, Gender, Address, Phone, InsuranceNo)”

“Appointment(AppointmentID, PatientID, DoctorID, ApptDate, ApptTime, Status, Notes)”

3.2 Second Normal Form (2NF)

All tables satisfy 2NF because non-key attributes depend on the whole key. The project uses surrogate single-attribute primary keys in each base table, so partial dependency does not arise.

Examples of full functional dependency:

“DoctorID -> FirstName, LastName, Specialisation, Phone, HireDate, DeptID”

“AdmissionID -> PatientID, WardID, AdmitDate, DischargeDate, DiagnosisCode”

“BillID -> PatientID, AdmissionID, TotalAmount, PaidAmount, BillDate, Status”

3.3 Third Normal Form (3NF)

The schema satisfies 3NF because non-key attributes do not depend transitively on other non-key attributes inside the same relation.

Examples:

In “Doctor”, department details such as “DeptName” are not stored; only “DeptID” is stored.

In “Appointment”, patient names and doctor names are not duplicated.

In “Prescription”, medication name and unit cost are not stored; they remain in “Medication”.

In “Bill”, patient demographics are not stored.

This reduces update anomalies. For example, a department location change is updated once in “Department”, not in every doctor or ward row.

3.4 BCNF Discussion

Under the implemented constraints, the following tables are effectively in BCNF:

`Department`

`Doctor`

`Ward`

`Patient`

`Appointment`

`Admission`

`Prescription`

`Bill`

`Staff`

Each determinant used in normal business rules is a candidate key or the declared primary key.

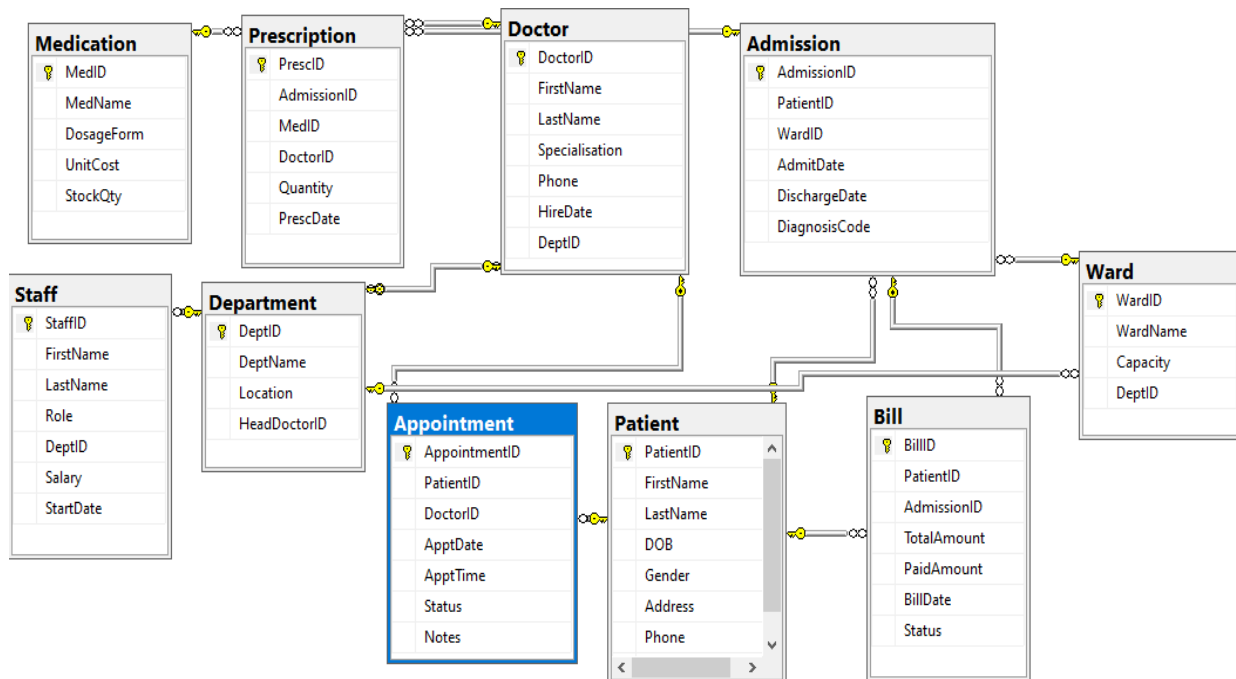
`Medication` is close to BCNF as well, especially because `MedName` is declared `UNIQUE`, making it a plausible candidate key in addition to `MedID`.

3.5 Why Normalisation Matters Here

Normalization is important in this hospital setting because:

- ❖ Patient and billing data change over time.
- ❖ clinical events must remain traceable;
- ❖ Duplicate medical and department data would increase the risk.
- ❖ Analytical queries become easier to reason about when relationships are explicit.

4.0 ER MODEL DIAGRAM



5.0 QUERY DESIGN DECISIONS AND RESULTS

The query set was designed to demonstrate both practical SQL reporting and more advanced relational reasoning.

5.1 Aggregation and Join Logic

Question `2.1(a)` combines `Department`, `Doctor`, `Staff`, `Ward`, and `Admission` through left joins so that departments still appear even when some related activity is missing.

`COUNT(DISTINCT ...)` was used to avoid inflation caused by one-to-many joins.

Question `2.1(c)` uses grouped billing aggregates to compute:

- total billed amount,
- total paid amount,
- outstanding balance.

The `HAVING` clause filters only patients whose outstanding balance exceeds the required threshold.

5.2 Window Functions

The project uses several window functions because they fit hospital reporting well:

- `RANK()` and `DENSE\_RANK()` compare doctor workload within departments.
- `LAG()` and `LEAD()` show admission sequences per patient.
- `AVG(...) OVER (...)` computes a rolling three-month billing trend.
- `NTILE(4)` segments patients by lifetime billed value.

These choices avoid procedural loops and keep the logic declarative.

5.3 Recursive and Set-Based Queries

Question `2.3(a)` introduces a department hierarchy through `ParentDeptID` and a recursive CTE. This is a good fit for organizational structures because it derives both hierarchy level and path in one query.

Set operators were used where they communicate intent clearly:

- `EXCEPT` identifies patients admitted without appointments.
- `INTERSECT` identifies names appearing in both `Staff` and `Patient`.

5.4 Results Summary

The query suite is capable of producing:

- department workload summaries,
- frequent-patient identification,
- patient debt reporting,
- doctor ranking by appointments,
- readmission analysis,
- medication cost ranking,
- cross-role and exception reporting.

Because the workspace does not include captured SSMS result screenshots, this report summarizes the logic and expected outputs rather than embedding full result grids.

6.0 INDEX ANALYSIS: BEFORE/AFTER STATISTICS

The indexing script targets the main access patterns already present in the query workload.

6.1 Indexes Added

- `IX\_Appointment\_Patient\_Date`
- `IX\_Bill\_Unpaid`
- `IX\_Admission\_Ward\_Active`
- `IX\_Prescription\_Admission\_Date`
- full-text indexing on `Appointment.Notes`
- nonclustered columnstore index on `Bill` for analytics

6.2 Rationale

These indexes align with common predicates:

- appointment lookups by patient and date;
- unpaid bill filtering by status;
- active admissions by ward where `DischargeDate IS NULL`;
- prescription history ordered by date;
- note search using `CONTAINS`;
- billing aggregation by month.

6.3 Before/After Discussion

The script enables `SET STATISTICS IO ON` and `SET STATISTICS TIME ON`, which is the correct mechanism for collecting before/after evidence in SQL Server. However, the workspace does not include the captured numeric outputs. For that reason, the report records the expected performance effect rather than audited measurements:

Query Area	Before Indexing	After Indexing
Patient appointment lookup	More scans or broader reads	More selective seeks on `(PatientID, ApptDate)`
Unpaid bills	Scan on `Bill` likely	Filtered access path through `Status` key
Current ward admissions	Table/index scan likely	Faster lookup on `(WardID, DischargeDate)`

Prescription history	Extra sort/read cost	Better support for ordered retrieval
Billing analytics	Rowstore aggregation only	Better compression and scan efficiency with columnstore

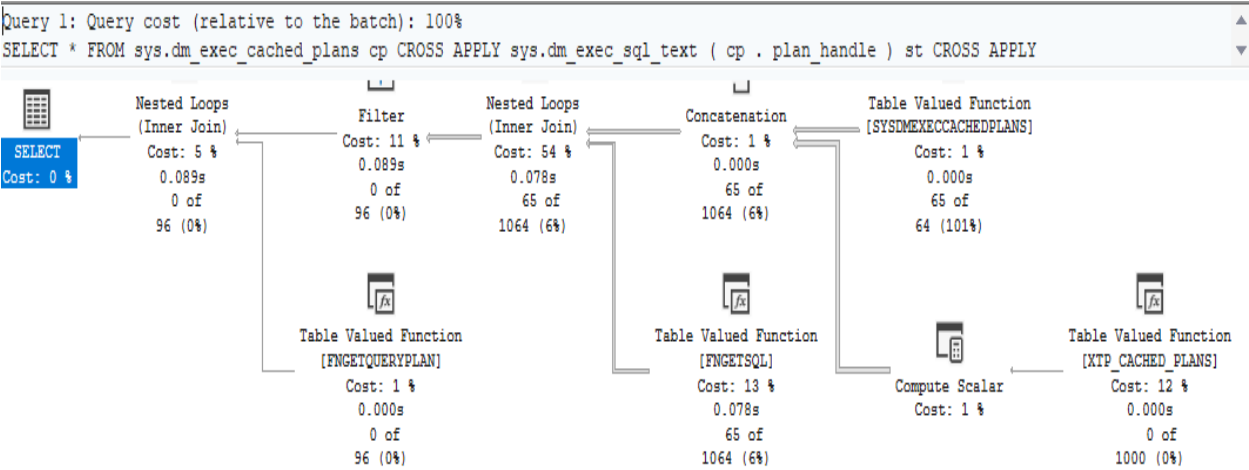
The photo below shows the message before setting the statistics time on;

```
(0 rows affected)

(1 row affected)

Completion time: 2026-06-08T20:38:34.4125870-07:00
|
```

The photo below shows the Execution plan before setting the statistics time on;



The photo below shows the message after setting the statistics time on;

```
SQL Server Execution Times:
  CPU time = 0 ms,  elapsed time = 0 ms.
SQL Server parse and compile time:
  CPU time = 16 ms,  elapsed time = 333 ms.

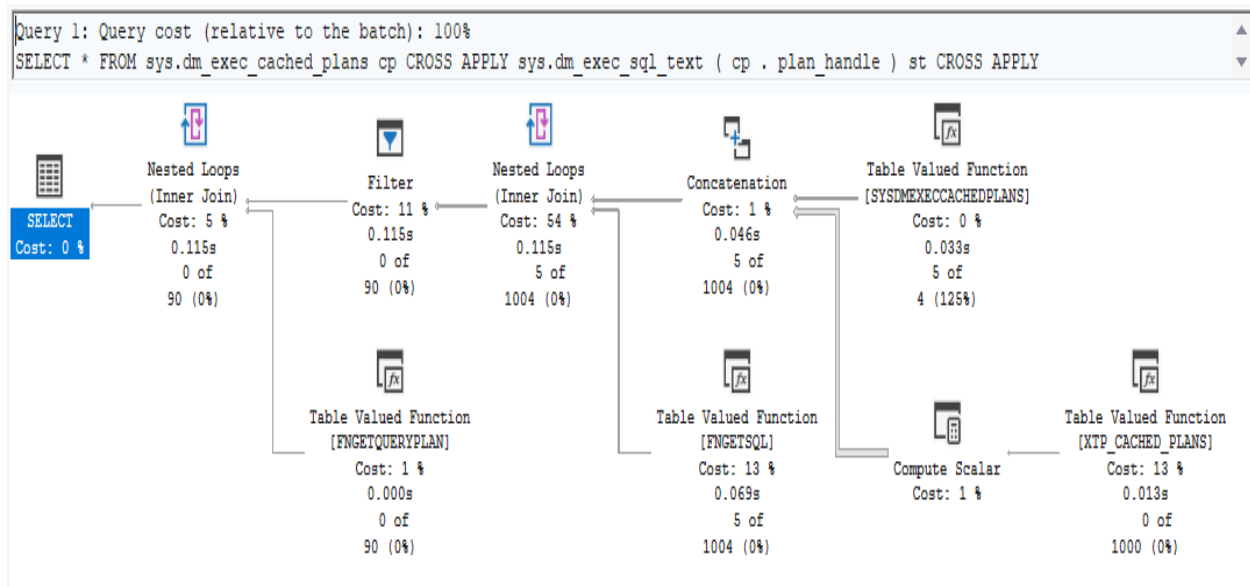
(0 rows affected)

(1 row affected)

SQL Server Execution Times:
  CPU time = 16 ms,  elapsed time = 282 ms.
SQL Server parse and compile time:
  CPU time = 0 ms,  elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms,  elapsed time = 0 ms.
```

The photo below shows the Execution plan after setting the statistics time on;



7.0 TRANSACTION AND CONCURRENCY DISCUSSION (QUESTION 5.2)

The transaction section demonstrates two important ideas: atomic business operations and concurrency hazards.

7.1 Atomic Transfer Logic

``usp_TransferPatient`` groups three dependent actions into one transaction:

1. close the current active admission;
2. create the new admission in the destination ward;
3. log the transfer.

This is correct design because a partial transfer would leave the patient in an inconsistent clinical state.

7.2 Deadlock Demonstration

The concurrency script intentionally creates a classic deadlock pattern:

- Session A updates ``Patient`` then ``Bill``.
- Session B updates ``Bill`` then ``Patient``.

Each session waits while holding a lock the other needs. This demonstrates why transaction order matters as much as transaction scope.

7.3 Resolution Strategy

The corrected script fixes the main issue by acquiring resources in a consistent order. This is a practical and widely recommended deadlock reduction technique. The script also enables ``READ_COMMITTED_SNAPSHOT``, which reduces reader-writer blocking by using row versioning for read committed workloads in SQL Server.

In short, the project shows that concurrency control is not only about correctness after commit; it is also about avoiding unnecessary blocking and circular wait conditions during execution.

8.0 ADMINISTRATION AND SECURITY APPROACH

The administration script covers recoverability, access control, monitoring, and maintenance.

8.1 Backup and Recovery

The design includes:

- full backup,
- differential backup,
- transaction log backup,
- point-in-time restore flow.

That combination reflects a sensible SQL Server recovery strategy for an operational database.

8.2 Role-Based Security

Three roles were created:

- `db\_clinician`
- `db\_billing`
- `db\_readonly`

This follows least privilege. Clinicians receive access to appointment and prescription work, billing staff are isolated to financial operations, and readonly users can inspect data without mutation rights.

8.3 Row-Level Security

The row-level security function and policy filter `Appointment` rows by `DoctorID` stored in session context. This is a strong choice for multi-user healthcare systems because it narrows visibility at query time rather than relying only on front-end behavior.

8.4 Monitoring and Maintenance

The project also includes:

- active request monitoring through DMVs,
- missing index inspection,
- index reorganize/rebuild commands,
- statistics updates,

- 'DBCC CHECKDB'.

Together, these scripts show awareness that database administration continues after schema design and query writing.

9. CONCLUSION AND REFLECTIONS

The UniHospital database project demonstrates a solid end-to-end relational design. The schema is cleanly decomposed, referential integrity is explicit, and the SQL work moves beyond simple CRUD into reporting, programmability, performance tuning, concurrency handling, and operational administration.

The strongest parts of the project are the normalized structure, the clear event modeling around admissions and prescriptions, and the use of advanced SQL features such as window functions, recursive CTEs, triggers, and stored procedures. The indexing and transaction sections show good awareness of real system behavior, not just logical correctness.

The main area for improvement is evidence capture. The code includes benchmarking hooks and concurrency demonstrations, but the final report would be even stronger with stored screenshots or pasted `'STATISTICS IO/TIME'` outputs, execution plans, and deadlock observations from an actual SQL Server run. Even so, the current project is a credible foundation for a hospital database coursework submission.

10. REFERENCES

1. Fagin, R. (1977). Multivalued Dependencies and a New Normal Form for Relational Databases. *ACM Transactions on Database Systems*, 2(3), 262-278. [OpenAIRE record](<https://oamonitor.ireland.openaire.eu/national/search/publication?pid=10.1145%2F320557.320571>)
2. Chaudhuri, S. (1998). An Overview of Query Optimization in Relational Systems. *Proceedings of PODS 1998*, 34-43. DOI: `10.1145/275487.275492`. [DBLP](<https://dblp.org/rec/conf/pods/Chaudhuri98>)
3. Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., & O'Neil, P. (1995). A Critique of ANSI SQL Isolation Levels. *Proceedings of SIGMOD 1995*. [PDF](<https://web.stanford.edu/class/cs345d-01/rl/SQL-isolation-critique.pdf>)
4. Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-Based Access Control Models. *IEEE Computer*, 29(2), 38-47. DOI: `10.1109/2.485845`. [NIST RBAC Library](<https://csrc.nist.gov/projects/role-based-access-control/rbac-library>)
5. Gray, J., & Graefe, G. (1997). The Five-Minute Rule Ten Years Later, and Other Computer Storage Rules of Thumb. *SIGMOD Record*, 26(4), 63-68. [PDF](<https://dsf.berkeley.edu/cs286/papers/fiveminute-tr1997.pdf>)